```
design_1_small.txt
Parsed 150 components, 120 nets, grid 52x52, 20 pins
Grid: Core sites by type: {0: 1500, 1: 600, 2: 300, 3: 100}
Initial HPWL: 6770
Final HPWL:   2185
Improvement:  4585 (67.7%)
Steps:        408
Time:         28.49s
---
design_2_medium.txt
Parsed 300 components, 250 nets, grid 52x52, 20 pins
Grid: Core sites by type: {0: 1500, 1: 600, 2: 300, 3: 100}
Initial HPWL: 13325
Final HPWL:   3467
Improvement:  9858 (74.0%)
Steps:        422
Time:         88.65s
---
design_3_large.txt
Parsed 500 components, 450 nets, grid 52x52, 20 pins
Grid: Core sites by type: {0: 1500, 1: 600, 2: 300, 3: 100}
Initial HPWL: 23464
Final HPWL:   6157
Improvement:  17307 (73.8%)
Steps:        434
Time:         202.24s
---
design_4_dense.txt
Parsed 800 components, 750 nets, grid 52x52, 20 pins
Grid: Core sites by type: {0: 1500, 1: 600, 2: 300, 3: 100}
Initial HPWL: 39084
Final HPWL:   10789
Improvement:  28295 (72.4%)
Steps:        444
Time:         451.51s
---
design_5_extreme.txt
Parsed 1200 components, 1100 nets, grid 52x52, 20 pins
Grid: Core sites by type: {0: 1500, 1: 600, 2: 300, 3: 100}
Initial HPWL: 57292
Final HPWL:   17651
Improvement:  39641 (69.2%)
Steps:        451
Time:         900.19s
```

Performance Results

Design	Cells	Nets	Initial HPWL	Final HPWL	Improvement	Time (s)	Moves/sec
Small	150	120	6770	2185	67.7%	28.49	215
Medium	300	250	13325	3467	74.0%	88.65	143
Large	500	450	23464	6157	73.8%	202.24	108
Dense	800	750	39084	10789	72.4%	451.51	79
Extreme	1200	1100	57292	17651	69.2%	900.19	55

Wire-length reduction is consistent (69-74%) but runtime significantly grows with design size (close to O(n^2) instead of O(n) as intended).

Identified Bottlenecks:

1.

Problem:

- Small (150 cells): around 28 sec -> Extreme (1200 cells): around 900 sec
- Throughput: 215 moves/sec -> 55 moves/sec (3.9x slowdown on 8x cell count)
- Probable cause: Move generation is O(n) (randomly searching through empty sites each iteration)

- generate_move() picks random cell then scans all same-type empty sites
- No caching of empty site lists, so it fully rebuilds after each move
- For dense designs, # of sites becomes large (especially type-0 -> 1500 sites) so random selection becomes slow

- If designs scale to 2000+ cells -> could exceed 30 minutes

2.

Problem:

- Algorithm spends around 40% of iterations at high T accepting mostly uphill moves
- Most improvement happens at T < T_init x 0.1 (bottom 10% of schedule)
- Suggests initial temperature is too high relative to early move gains

Evidence from temperature vs. cost traces:

- Design 1: cost improves from 6770 -> around 5000 (by T=10%) then plateaus and drifts upward
- Design 5: similar pattern as cost oscillates ±500 for long stretches at high T

cause: T_init = 500 * initial_cost from spec doesn't account for site-type constraints

3.

Problem:

- At T < 1, acceptance rate near 0; most rejected moves
- Algorithm essentially stops exploring
- Stuck in local minima

Example (Design 5):

- T drops from around 10 to around 0.01 (120+ iterations)
- Cost stays 19500-20000 (no progress)
- Only last 1% of time yields improvement

Weaknesses:

- empty_by_type rebuilt manually after each move (inefficient)
- no per-design tuning of cooling rate or temperature schedule
- no early-stopping criterion (runs full schedule regardless of convergence)

Bottlenecks for runtime:

- recompute HPWL for all affected nets from scratch each proposed move.
    - This is O(net_size) per affected net and can dominate runtime for large nets and large designs.
- ‘generate_move()’ rebuilds the same-type candidate list every move by iterating over all components.
    - This is O(#cells) per move and is inefficient for hundreds or thousands of cells.
- ‘best_positions’ saves all component coordinates on every improvement.
    - This can be expensive when improvements happen often in early annealing.

Bottlenecks for optimizing wirelength

- Initial placement is fully random within legal type sites.
    - depends heavily on random seed and often starts far from a good solution.
- Move selection is entirely random among same-type cells or empty sites.
- Cost function uses plain HPWL with no congestion awareness.
    - limits the algorithm to a basic wire-length objective
- No multi-cell aware moves.
    - Only single-cell moves or pairwise swaps are allowed -> slower convergence.